

Comparação de Algoritmos de Filtragem Colaborativa sobre a Base MovieLens 100k

Mariana Ferreira Ferrarez Ribeiro
Universidade Federal Rural do Rio de Janeiro
Rio de Janeiro, Brasil

Rafael Vieira da Silva
Universidade Federal Rural do Rio de Janeiro
Rio de Janeiro, Brasil

RESUMO

Este trabalho apresenta a implementação e comparação de oito algoritmos de filtragem colaborativa, divididos em abordagens baseadas em memória e baseadas em modelo. Os algoritmos foram avaliados sobre a base MovieLens 100k utilizando métricas de predição de rating (RMSE, MAE) e de qualidade de recomendação (Precision@10, Recall@10, NDCG@10, MAP@10). A hiperparametrização foi conduzida via grid search com validação cruzada de 5 folds, e uma análise de sensibilidade foi realizada para investigar o impacto dos principais hiperparâmetros no desempenho. Os resultados mostram que o SVD Regularizado obteve o melhor desempenho em predição de rating (RMSE = 0,933), enquanto o BPR apresentou os melhores resultados em métricas de ranking (Precision@10 = 0,072).

1 INTRODUÇÃO

O crescimento exponencial do comércio eletrônico e das plataformas de conteúdo digital tem gerado uma sobrecarga de informações que dificulta a tomada de decisão dos usuários. Nesse cenário, os Sistemas de Recomendação (SR) têm desempenhado um papel fundamental ao auxiliar os consumidores a encontrar produtos e serviços relevantes de forma personalizada [?].

Grande parte das pesquisas em SR concentra-se na melhoria dos algoritmos de predição para aprimorar a acurácia das recomendações [?]. A competição Netflix Prize demonstrou que havia oportunidades significativas para melhorar os algoritmos existentes, impulsionando pesquisas em técnicas como fatoração matricial, que se tornaram estado da arte [?].

O objetivo deste trabalho, desenvolvido no âmbito da disciplina *Introdução à Teoria de Sistemas de Recomendação*, ministrada pelo Prof. Filipe Braidão na UFRRJ, é implementar e comparar diferentes algoritmos de filtragem colaborativa, incluindo quatro abordagens baseadas em memória e quatro baseadas em modelo. Além da comparação direta, investimos a hiperparametrização de cada algoritmo por meio de grid search com validação cruzada de 5 folds, e realizamos uma análise de sensibilidade para avaliar o impacto dos principais hiperparâmetros no desempenho dos modelos. O código-fonte está disponível publicamente¹.

2 EXPERIMENTOS

2.1 Base de Dados

Utilizamos a base MovieLens 100k², que contém 100.000 avaliações (ratings de 1 a 5) de 943 usuários sobre 1.682 filmes. As avaliações são esparsas — cada usuário avaliou, em média, cerca de 106 filmes, o que corresponde a uma densidade de aproximadamente 6,3%.

Para a avaliação final, utilizamos a partição `ua.base` (90.570 avaliações para treino) e `ua.test` (9.430 avaliações para teste), fornecidas pelo próprio dataset. Para a hiperparametrização e análise de sensibilidade, utilizamos os 5 folds pré-definidos (`u1-u5`), cada um com uma divisão 80/20.

2.2 Algoritmos Implementados

Todos os algoritmos foram implementados do zero em Python, sem uso de bibliotecas externas de recomendação. A seguir, descrevemos brevemente cada um.

2.2.1 Baseados em Memória.

- **Simple Memory:** baseline que combina a média do usuário e a média do item com peso α , e recomenda itens por popularidade. A predição é dada por $\hat{r}_{ui} = \alpha \cdot \bar{r}_u + (1 - \alpha) \cdot \bar{r}_i$.
- **User-Based kNN:** calcula a similaridade cosseno entre vetores de ratings centrados pela média do usuário. A predição utiliza a média ponderada dos desvios dos k vizinhos mais similares que avaliaram o item.
- **Item-Based kNN:** utiliza a similaridade cosseno ajustada (adjusted cosine) entre itens, subtraindo a média do usuário antes do cálculo. A predição é análoga ao user-based, mas sobre vizinhos de itens.
- **Slope One:** calcula desvios médios entre pares de itens e prediz com base nas avaliações do usuário para outros itens, ponderando pela contagem de co-ocorrências com suavização (shrinkage): $w_{ij} = \frac{c_{ij}}{c_{ij} + \lambda}$, onde λ é o fator de shrinkage.

2.2.2 Baseados em Modelo.

- **Simple Model:** modelo de biases que decompõe o rating como $\hat{r}_{ui} = \mu + b_u + b_i$, onde μ é a média global, b_u é o bias do usuário e b_i é o bias do item, estimados com regularização λ .
- **SVD Regularizado:** fatoração matricial com fatores latentes treinada via SGD: $\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$. Os parâmetros são otimizados minimizando o erro

¹<https://github.com/ravsil/sistemasDeRecomendacao>

²<https://grouplens.org/datasets/movielens/100k/>

quadrático com regularização L2 sobre biases e fatores [?].

- **BPR (Bayesian Personalized Ranking)**: otimiza um critério pairwise para ranking, maximizando a diferença de score entre itens positivos (rating ≥ 4) e negativos amostrados aleatoriamente. Foi projetado para feedback implícito, não para predição de rating.
- **Factorization Machines (FM)**: generaliza a fatoração matricial usando features one-hot de usuário e item. Modela interações de segunda ordem entre features de forma eficiente: $\hat{y} = w_0 + \sum_i w_i x_i + \sum_{i < j} \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$. Treinado via SGD com regularização L2.

2.3 Métricas de Avaliação

Avaliamos os algoritmos em duas dimensões complementares:

Predição de rating: mede a qualidade da estimativa numérica da nota.

- **RMSE** (Root Mean Squared Error): penaliza erros maiores de forma quadrática, sendo mais sensível a outliers.
- **MAE** (Mean Absolute Error): fornece o erro médio absoluto, mais interpretável e robusto a outliers que o RMSE.

Utilizamos ambas as métricas pois são complementares: o RMSE é o padrão da literatura para comparação [?], enquanto o MAE oferece uma interpretação mais direta do erro médio.

Qualidade de recomendação (top-K): mede a qualidade da lista ordenada de recomendações, considerando como relevantes os itens com rating $\geq 4,0$ no conjunto de teste.

- **Precision@K**: fração dos itens recomendados que são relevantes.
- **Recall@K**: fração dos itens relevantes que foram recomendados.
- **NDCG@K**: considera a posição dos acertos, atribuindo maior importância a itens relevantes no topo da lista.
- **MAP@K**: média da precisão nos pontos de acerto, ponderada pela posição.

As métricas de ranking são importantes pois, na prática, o usuário visualiza apenas os primeiros itens recomendados, tornando a ordenação tão relevante quanto a acurácia pontual.

2.4 Hiperparametrização

A hiperparametrização foi realizada via **grid search com validação cruzada de 5 folds**, utilizando as partições pré-definidas do MovieLens 100k (u1–u5). Para cada combinação de hiperparâmetros, o algoritmo foi treinado e avaliado em cada fold, e o RMSE médio ($\pm$ desvio padrão) foi registrado.

A Tabela ?? apresenta os hiperparâmetros investigados e os melhores valores encontrados para cada algoritmo.

Tabela 1: Hiperparâmetros investigados e melhores valores encontrados via validação cruzada de 5 folds.

Algoritmo	Parâmetro	Faixa testada	Melhor
Simple Mem.	α (peso)	0,0 a 1,0	0,4
User-Based	k (vizinhos)	5 a 80	30
Item-Based	k (vizinhos)	5 a 80	30
Slope One	λ (shrink)	0 a 150	100
Simple Mod.	λ (reg.)	10^{-7} a 25	10
SVD	d, η, λ	10–50, 0,005–0,01, 0,01–0,05	50; 0,01; 0,01
BPR	d, η, λ	10–50, 0,01–0,05, 0,0025–0,01	50; 0,05; 0,0025
FM	$d, \eta, \lambda_w, \lambda_v$	5–20, 0,005–0,01, 0,001–0,01	20; 0,01; 0,001; 0,001

2.5 Resultados

2.5.1 Predição de Rating. A Tabela ?? apresenta os resultados da avaliação final, utilizando os melhores hiperparâmetros encontrados.

Tabela 2: Resultados da avaliação final com hiperparâmetros otimizados (partição ua.base/ua.test).

Algoritmo	RMSE	MAE	P@10	R@10	NDCG@10	MAP
Simple Memory	0,995	0,804	0,083	0,148	0,132	0,083
User-Based	0,959	0,750	0,002	0,003	0,002	0,002
Item-Based	0,956	0,747	0,010	0,019	0,013	0,008
Slope One	0,961	0,756	0,002	0,003	0,004	0,002
Simple Model	0,976	0,774	0,033	0,058	0,047	0,025
SVD	0,933	0,734	0,037	0,065	0,060	0,028
BPR	2,256	1,900	0,072	0,127	0,117	0,055
FM	0,959	0,759	0,032	0,055	0,039	0,018

Em termos de predição de rating, o **SVD Regularizado** obteve o melhor desempenho com RMSE de 0,933 e MAE de 0,734. O **FM** obteve RMSE de 0,959, próximo dos métodos baseados em memória. Os resultados confirmam a superioridade da fatoração matricial com biases (SVD) para esta tarefa, em linha com os achados da literatura [?].

Entre os métodos baseados em memória, o **Item-Based kNN** obteve o menor RMSE (0,956), levemente melhor que o User-Based (0,959) e o Slope One (0,961). O **Simple Memory** obteve o pior resultado entre os métodos de memória (0,995), o que é esperado por ser um baseline simples.

O **BPR** apresentou RMSE muito alto (2,256) porque foi projetado para otimizar ranking via feedback implícito, não para predizer ratings. Isso é esperado e não representa uma falha do algoritmo, mas sim uma inadequação da métrica para este tipo de modelo.

2.5.2 Qualidade de Recomendação. Em métricas de ranking (Precision@10, Recall@10, NDCG@10, MAP@10), o cenário se inverte: o **BPR** obteve os melhores resultados em todas as métricas de ranking (Precision@10 = 0,072, Recall@10 = 0,127), demonstrando que a otimização pairwise é eficaz para gerar listas de recomendação de qualidade.

O **Simple Memory** obteve o segundo melhor Precision@10 (0,083), o que se explica pela sua estratégia de

recomendar itens populares — uma heurística surpreendentemente competitiva. O **SVD** ficou em terceiro (0,037), mostrando que a fatoração matricial também produz boas listas de recomendação.

Notavelmente, os métodos kNN (User-Based e Item-Based) obtiveram métricas de ranking muito baixas, apesar de boas métricas de predição. Isso sugere que a acurácia na predição pontual de ratings não se traduz necessariamente em boas listas de recomendação.

2.5.3 Visualizações. As Figuras ?? e ?? apresentam, respectivamente, os scatter plots (rating real vs. predito) e as curvas ROC para quatro algoritmos representativos. A Figura ?? mostra as matrizes de confusão correspondentes, considerando o limiar de relevância em 4,0.

2.5.4 Análise de Sensibilidade. A análise de sensibilidade investigou como a variação de um hiperparâmetro-chave impacta o desempenho (RMSE) de cada algoritmo, mantendo os demais fixos nos melhores valores. Os principais achados foram:

Número de vizinhos k (User-Based e Item-Based): ambos os algoritmos apresentaram uma curva em forma de U, com RMSE diminuindo rapidamente de $k = 5$ até $k = 20$ – 30 , estabilizando e depois aumentando levemente para $k > 50$. O ponto ótimo $k = 30$ indica um equilíbrio entre incluir vizinhos informados e evitar ruído de vizinhos pouco similares.

Número de fatores latentes (SVD, BPR, FM): para os três modelos, mais fatores latentes resultaram em melhor desempenho, com ganhos decrescentes. O SVD passou de 0,928 ($d = 5$) para 0,911 ($d = 100$), uma melhoria de 1,8%. O FM apresentou comportamento similar, mas com ganho menos monotônico.

Shrinkage λ do Slope One: o RMSE diminuiu monotonicamente com o aumento de λ (de 0,948 sem shrinkage para 0,943 com $\lambda = 150$), indicando que a suavização por contagem é benéfica e que pares com poucos co-raters devem ser penalizados.

Peso α do Simple Memory: a curva mostrou um formato de U simétrico com mínimo em $\alpha = 0,4$, indicando que dar peso ligeiramente maior à média do item (60%) do que à do usuário (40%) é ótimo. Isso sugere que a qualidade intrínseca do item é um preditor ligeiramente mais informativo que o padrão geral do usuário.

Regularização do Simple Model: o RMSE decresceu até $\lambda = 10$ e depois voltou a subir para $\lambda = 25$, mostrando uma curva em U típica do trade-off entre viés e variância.

As Figuras ?? a ?? ilustram graficamente essas tendências.

3 CONCLUSÃO

Este trabalho implementou e comparou oito algoritmos de filtragem colaborativa, divididos entre abordagens baseadas em memória e baseadas em modelo, sobre a base MovieLens 100k.

Os principais achados foram: (1) o **SVD Regularizado** obteve o melhor desempenho em predição de rating (RMSE = 0,933), confirmando a eficácia da fatoração matricial; (2) o

BPR superou todos os outros em métricas de ranking, demonstrando a importância de otimizar diretamente para a tarefa desejada; (3) a acurácia na predição de rating não implica necessariamente em boas recomendações, como evidenciado pelos métodos kNN; e (4) a hiperparametrização produziu melhorias consistentes, com o SVD reduzindo seu RMSE de 0,958 (default) para 0,933 (otimizado), uma melhoria de 2,6%.

A análise de sensibilidade revelou padrões interpretáveis: o número ideal de vizinhos para métodos kNN é finito ($k \approx 30$), enquanto modelos de fatoração se beneficiam de mais fatores latentes com retornos decrescentes.

Como sugestões para trabalhos futuros, destacamos: (a) explorar técnicas de ensemble combinando modelos de predição e de ranking; (b) avaliar os algoritmos em bases maiores (MovieLens 1M, CiaoDVD) para testar a escalabilidade; (c) incorporar informações contextuais e de conteúdo para melhorar as recomendações cold-start; e (d) investigar técnicas de pré-processamento como normalização de ratings e remoção de outliers.

REFERÊNCIAS

- [1] G. Adomavicius and A. Tuzhilin. Toward the next generation of recommender systems: a survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6):734–749, jun 2005. ISSN 1041-4347. doi: 10.1109/TKDE.2005.99.
- [2] Y. Koren, R. Bell, and C. Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- [3] F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor. *Recommender Systems Handbook*. Springer, 2011. doi: 10.1007/978-0-387-85820-3.

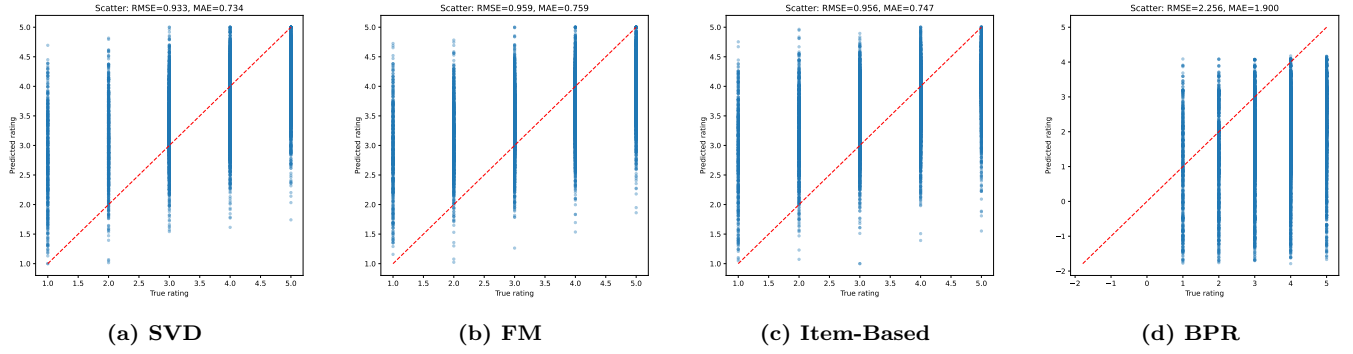


Figura 1: Scatter plots: rating real vs. predito.

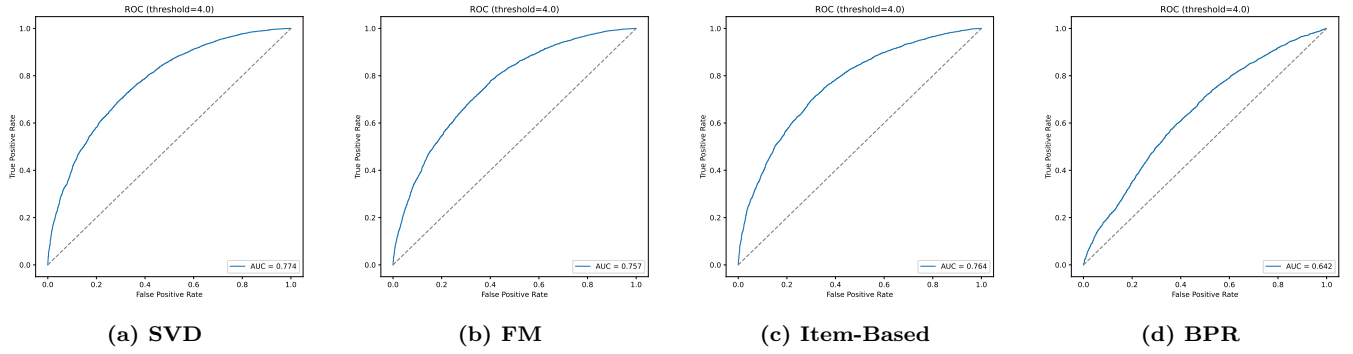


Figura 2: Curvas ROC (limiar = 4,0).

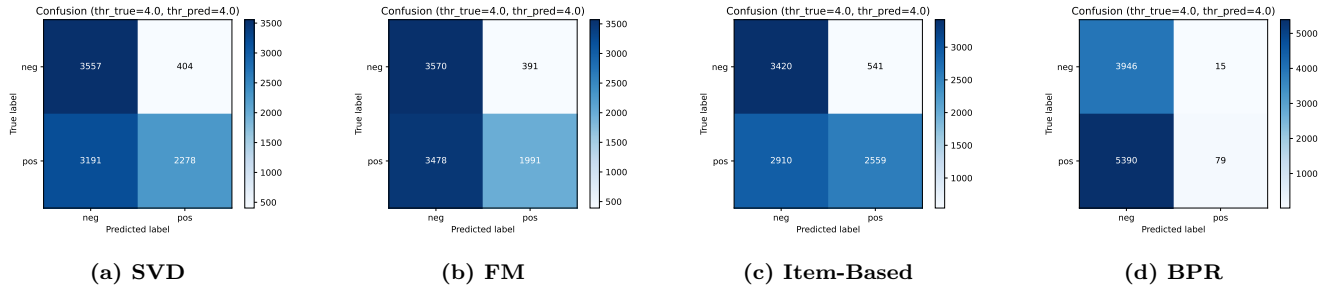


Figura 3: Matrizes de confusão (limiares = 4,0).

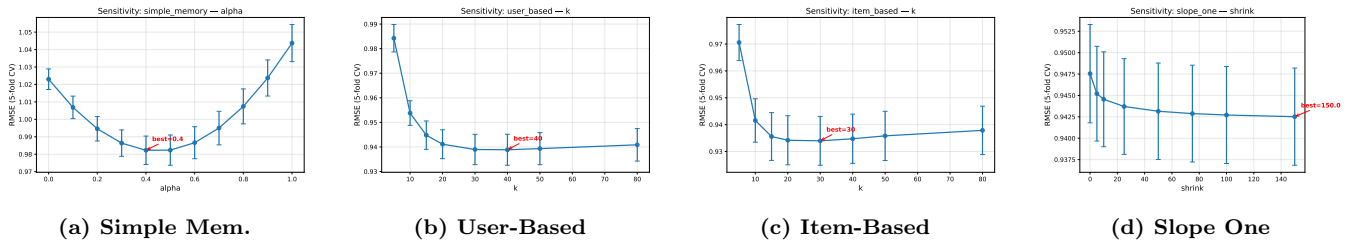


Figura 4: Sensibilidade – algoritmos baseados em memória.

Comparação de Algoritmos de Filtragem Colaborativa sobre a Base MovieLens 100k

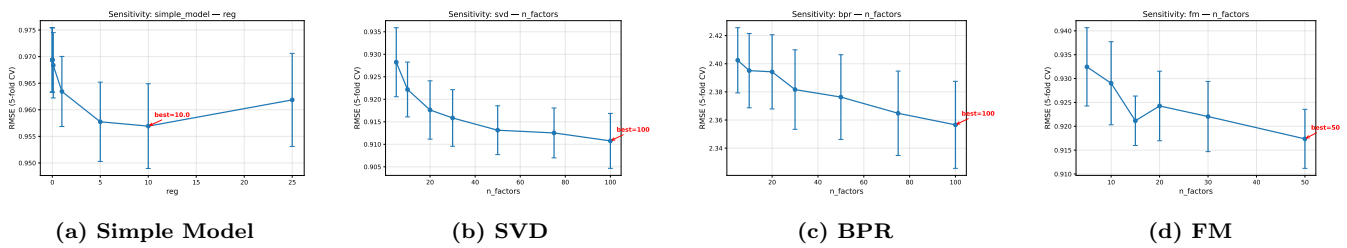


Figura 5: Sensibilidade – algoritmos baseados em modelo.

Temporary page!

L^AT_EX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because L^AT_EX now knows how many pages to expect for this document.